Search-Based Software Re-Modularization: A Case Study at Adyen

Casper Schröder, Adriaan van der Feltz
Adyen N.V.
Amsterdam, the Netherlands
{casper.schroder, adriaan.vanderfeltz}@adyen.com

Annibale Panichella, Maurício Aniche
Delft University of Technology
Delft, the Netherlands
{A.Panichella, M.FinavaroAniche}@tudelft.nl

***Abstract*—Deciding what constitutes a single module, what classes belong to which module or the right set of modules for a specific software system has always been a challenging task. The problem is even harder in large-scale software systems composed of thousands of classes and hundreds of modules. Over the years, researchers have been proposing different techniques to support developers in re-modularizing their software systems. In particular, the search-based software re-modularization is an active research topic within the software engineering community for more than 20 years.**

This paper describes our efforts in applying search-based software re-modularization approaches at *Adyen*, a large-scale payment company. *Adyen*'s code base has 5.5M+ lines of code, split into around hundreds of modules. We leveraged the existing body of knowledge in the field to devise our own search algorithm and applied it to our code base. Our results show that search-based approaches scale to large code bases as ours. Our algorithm can find solutions that improve the code base according to the metrics we optimize for, and developers see value in the recommendations. Based on our experiences, we then list a set of challenges and opportunities for future researchers, aiming at making search-based software re-modularization more efficient for large-scale software companies.

***Index Terms*—software engineering, search-based software engineering, software refactoring, software modularization.**

I. Introduction

The existing body of knowledge on how to structure a software system in such a way that its evolution and maintenance can be done in effective and sustainable ways is extensive; it ranges from the canonical works by Parnas on software modularization [1], to Booch et al.'s view on object-oriented software design [2], to modern software design techniques, such as Domain-Driven Design (DDD) [3], to microservices [4]. Nevertheless, designing large-scale software systems is still an important and practical challenge for software development teams.

In this paper, we highlight the challenges of modularizing a large-scale software system. Modularity is the idea of partitioning a (often large) software system into different components to reduce its overall complexity [5]. However, deciding what constitutes a single module, what classes belong to which module or the right set of modules for a specific software system is challenging. The challenge might be even harder at the initial phases of the software development process, as little is known about the application itself [2]. Finally, the way contemporary large software companies (such as *Adyen*, the industry partner that serves as a case study in this research) work and divide the development responsibilities also augment the challenge: modules evolve in fast paces, are developed by different software teams, and any possible intersections between modules (e.g., does this responsibility belong to module A or B?) are solved ad-hoc.

Modularization of code has been a software engineering research topic for more than 20 years. We see works focusing on, e.g., proposing code metrics that aim at detecting modules that are not cohesive or are too much coupled (e.g., [6, 7, 8]), and automated detection of modularization anti-patterns (e.g., [9]). Closely related to this research, we also see a large effort from the community in modeling software (re-)modularization as an optimization problem and employing search-based techniques to find candidate solutions (e.g., [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]). The different studies have relied on different software quality metrics, e.g., coupling, cohesion, number of modules and clusters, effort, package size, number of changes, and semantic cohesiveness. They also use different optimization algorithms, e.g., hill climbing, vanilla GA, NSGA-II, interactive GA, NSGA-III, and MOEA. However, while current results are promising and the accuracy of these techniques has been evolving over time, we still only have little evidence of how these techniques can work in large-scale enterprise software systems.

In this paper, we describe an industrial case study on the effectiveness of search-based approaches for software re-modularization at *Adyen*, a large-scale enterprise software system composed of millions of lines of code and hundreds of developers. We also proposed a new quality metric (search objective) that takes transitive coupling and building time as part of the search problem, an objective that is perceived as fundamental in practice by the *Adyen* developers. We present our findings in two phases. First, we analyzed the improvements we obtain by looking at the solutions in the Pareto front. Second, we presented a set of the results to different developers, experts in the modules that the improvement was suggested and collect their perception.

Our results show that (i) existing search-based approaches scale to code bases with similar sizes (5.5M+ lines of code), (ii) solutions present significant improvements as measured by the optimization metrics used as search objectives, (iii) the novel metric that captures transitive dependencies and their

impact on module caching is a useful metric for the re–modularization problem, and (iv) developers see value in the recommended refactoring operations. However, while the generated solutions identify classes in wrong modules often accurately, the recommendation of which module to move it to is often imprecise.

This paper makes the following contributions:

1) A case study on the effectiveness of multi-objective evolutionary optimization approaches to the software (re-)modularization problem on a large-scale enterprise software system.
2) A new optimization quality metric (search objective) that takes into consideration transitive dependencies and build cost that can be used as part of the search process for better (re-)modularization solutions.
3) A set of challenges and opportunities for future work on search-based software re-modularization, derived from our experiences at *Adyen*.

II. Related Work

In Table I, we list the different papers in the search-based re-modularization line of research that inspired us, grouped by the algorithms they apply and the metrics they optimize for.

Overall, we observe a vast mixture of different algorithms being applied, with vanilla genetic algorithms and the NSGA family being among the most popular ones. We also observe that coupling and cohesion metrics are often measured in different ways and are the most commonly used objectives.

Meta-studies in software refactoring seem to observe similar trends. Ebad and Ahmed [24] compare different approaches to the modularization problem. The authors created a framework that measures approaches based on the given packaging goal, underlying principle, input artifact, internal quality attribute, search algorithm, fitness function, scalability, soundness, practicality, and supportability. They observe the following aspects: 1) most research sees packaging as an optimization problem, 2) Most research maximizes intra-package cohesion and minimize inter-package coupling, 3) most research uses genetic algorithms as their search methods, 4) all approaches are graph-based, 5) most use source code as the input artifact, 6) the selection of parameters of the heuristic search method is very crucial (due to local optima), 7) most approaches are scalable, 8) papers do not often provide tools that are easy to use (according to authors, only a single paper fit this criteria)

In this paper, we take advantage of the already extensive body of knowledge in the field, and apply it to a large-scale enterprise system. More specifically, we: (i) evaluate whether the approaches proposed by the paper are indeed scalable, by applying it to a 5.5M+ code base (we note that most research deals with projects in the scale of thousands of lines of code, not millions of lines of code), (ii) measure the perception of developers regarding the recommendations (we note that most research currently evaluates their results by means of either controlled or quantitative experiments, but not with the actual developers of the software systems).

TABLE I: Related work in search-based software re-modularization, grouped according to their algorithm and optimization variables of choice.

Search algorithms	
K-cut	Jermaine [25]
Clustering	Lee et al. [26]
Hill climbing	Mitchell [27], Mitchell and Mancoridis [12, 28], Marx et al. [29]
Genetic Algorithm	Harman et al. [22], Mitchell [27], Mitchell and Mancoridis [12, 28], Kwong et al. [10], Praditwong et al. [13]
Min-cut algorithms	Marx et al. [29]
NSGA-II	Ouni et al. [16], Abdeen et al. [15], Mahouachi [18]
Interactive NSGA-II	Bavota et al. [20]
NSGA-III	Mkaouer et al. [30]
Exhaustive search	Marx et al. [29]
Search objectives	
Cohesion (intra-edges)	Jermaine [25], Harman et al. [22], Mitchell [27], Mitchell and Mancoridis [12, 28], Kwong et al. [10], Praditwong et al. [13], Mahouachi [18]
Cohesion (functionality similarity)	Lee et al. [26]
Cohesion (Common Closure Principle, Common Reuse Principle)	Abdeen et al. [15]
Coupling (inter-edges)	Jermaine [25], Harman et al. [22], Mitchell [27], Mitchell and Mancoridis [12, 28], Kwong et al. [10], Praditwong et al. [13], Mahouachi [18]
Coupling (any type of interaction)	Lee et al. [26]
Coupling (Acyclic Dependencies Principle)	Abdeen et al. [15]
Number of modules	Harman et al. [22]
Functional performance	Kwong et al. [10]
Number of broken dependencies	Marx et al. [29]
Characteristics of the clusters	Praditwong et al. [13]
Semantic similarity	Ouni et al. [16], Mkaouer et al. [30], Mahouachi [18]
Method calls and field accesses	Ouni et al. [16]
Effort required	Bavota et al. [20], Mahouachi [18]
Package size	Abdeen et al. [15]
Number of required changes	Abdeen et al. [15], Mkaouer et al. [30]

III. The Case Study

In this section, we give an overview of the business, size, and scale of our industry partner, and we list the challenges and assumptions that emerge when applying search-based software re-modularization techniques to the case study.

A. *Our industry partner:* Adyen

Adyen is a payment service provider, gateway, acquirer, and point of sale terminal service[1]. The company was founded in 2006 and has grown rapidly, totaling over 1,400 employees as of August 2020. Most of the code is part of a monolithic repository, consisting of 5.5M+ lines of code. Due to the emphasis on uptime, performance, and growth potential for this type of business, good code quality, and code structure quality are essential. A code base of this size and speed of

[1]https://www.adyen.com/

growth brings additional challenges when compared to smaller code bases.

B. Definition of modules

Adyen's code base is divided into around hundreds of modules, each module having a different business responsibility. In this paper, a module can be interpreted as a package comprised of one or more Java classes. In this paper, we aim at identifying classes that currently reside in the wrong module, and recommend which module to move the class to.

C. Challenges of the scale

Size and complexity of the code base. The most obvious challenge coined by this case study is the size and complexity of the code base. As we mentioned, *Adyen*'s code base is composed of 5.5M+ lines of code, split into hundreds of modules. This implies a natural increase in the input that the chosen algorithm will take and its time to find the optimal output.

The impact of transitive dependencies. The large number of different modules emphasizes another aspect: the impact of transitive coupling. The transitive coupling can be described as follows: if a given module M1 depends on a given module M2, and module M2 depends on given module M3, M1 is, therefore, transitively dependent on M3. Given the large number of modules and their existing dependencies, any refactoring that is suggested by the approach may profoundly impact the transitive dependencies. Moreover, moving a class from one module to the other may even affect build and compilation time. For example, suppose that the tool recommends some class C to move from module A to module B. All modules that depend on class C will now have to depend on module B. For some modules, this will mean an extra dependency. If C is a class that changes very often, module B (and all its recursive dependencies) will need to be re-compiled often. Companies that develop large-scale software systems like *Adyen* often rely on module caching to avoid the recompilation of the entire code base; any refactoring suggestion should therefore take the compilation time, and cache breaks into consideration. As we explain before, we use transitive dependencies and cache breaking as an optimization variable.

A full refactoring is not possible in practice. Approaches that suggest completely new ways of modularizing the entire software system are bound not to be used at *Adyen*. Given the already large size of the code base, a full (or large) refactor is considered highly risky. Thus, any proposed approach should be able to propose refactoring solutions in small doses, i.e., they should be as small as possible.

IV. The approach

In this section, we discuss the design decisions we made in the search algorithm we propose to generate re-modularization refactoring suggestions at *Adyen*. In a nutshell, we formalize the re-modularization problem as follows:

Definition 1. *Let $C = \{C_1, \ldots, C_n\}$ be the set of classes in a software system, and let $M = \{M_1 \ldots, M_k\}$ be the set of existing software modules. Find a set of move class operations $X = \{C_i \rightarrow M_j, \ldots, C_h \rightarrow M_k\}$, $C_i \rightarrow M_j$ denoting that the class C_i is moved to the module M_j, that optimizes the following four search objectives:*

$$\begin{cases} minf_1(X) = \text{coupling}(C, M, X) \\ maxf_2(X) = \text{cohesion}(C, M, X) \\ minf_3(X) = |X| \\ minf_4(X) = \text{cost-of-transitive-dependencies}(C, M, X) \end{cases} \tag{1}$$

with the constraint that there is no circular dependencies across the modules M after the refactoring (as otherwise it is not possible to build the software).

In the following sub-sections, we describe the problem representation, the search objectives, and the optimization algorithm we used in our approach in detail.

A. Modeling & Representation

We encode a solution (*chromosome*) as a set of move-class operations of variable lengths. More precisely, a solutions is a set of move-class operations $S = \{C_i \rightarrow M_j, ..., C_h \rightarrow M_k\}$, where a generic operation $C_i \rightarrow M_j$ denotes that the class C_i is moved to the model M_j (move-class operations). Note that, in this representation, a chromosome has variable size, i.e., the number of move-class operations can vary during the search (by adding or removing operations).

The choice to model the solutions as changes instead of a complete dependency structure (as done by Harman et al. [22]) was made to reduce the cost of evaluating the solution (fitness evaluation). Re-evaluating entire new proposals on how to modularize a software system takes significantly more processing power than solely evaluating a set of changes (i.e, the diffs). In our experience, representing solutions as a set of changes also dramatically reduces the memory consumption. Besides, modeling solutions as set of changes has been previously proposed by Abdeen et al. [15] and also applied by other researchers in the field [18, 14].

B. Search Objectives

We optimize the following four search objectives:

Coupling. This objective is often proposed as code quality measures [31, 32, 25, 33, 34, 35, 36], and is often used in combination with search algorithms to improve the code quality of a code base (e.g., [15, 16, 18, 13, 30, 12, 37, 22, 38, 20, 6, 21]). In this paper, we measure software coupling using the *inter-module coupling* (InterMD). InterMD measures the total number of dependencies modules have with the other modules of the system. A dependency between two modules A and B happen whenever any class from module A depends on any class from module B. The overall InterMD of an individual is therefore measured as the sum of the module dependencies for all modules in the system.

Cohesion. We use the *intra-module coupling* (IntraMD) as the cohesion metric. IntraMD measures the number of intra-module dependencies (i.e., number of dependencies among classes within the same module) divided by the maximum possible amount of intra-dependencies, similar to Harman et al. [22]. We have experimented with two other metrics, based on the Common Closure Principle (CCP) and the Common Reuse Principle (CRP) [39]. Our preliminary analysis showed better improvements when using IntraMD, and therefore, it is the one we report in this paper.

Number of changes. This objective measures the amount of "move-class" refactorings, i.e. the number of classes that have been moved to a different module in the solution. Therefore, given two solutions X_1 and X_2 achieving the same InterMD and IntraMD, we prefer the solution with fewer changes. This objective also reflects the *manual effort* needed by developers to manually validate the generated solutions (refactoring recommendations) and eventually accept the changes. As we state before, many improvements can be made to a real-world large-scale code base; however, the effort needed might not be worth the gain in quality, nor companies are ready to risk large numbers of changes at once.

Cost of module cache breaks. As mentioned before, the transitive coupling is an important and critical objective to consider for our industrial partner. *Adyen* uses module caching to avoid the recompilation of the entire code base. Caching is often used to remedy the fact that building code after every change can take a long time. Instead of re-compiling all the software system modules, *Adyen* only re-compiles modules that changed and their dependencies, recursively. This causes any change to have a cascading effect throughout the code base, in the reverse direction of the module dependencies. Removing a single dependency or splitting a module significantly effect on the number of caches broken. To the best of our knowledge, no prior work considered the cost of module cache breaks.

An adequate re-modularization should minimize the re-build time needed due to transitive coupling by reducing the number of module cache breaks. In theory, the simplest solution would be to directly measure the building time (with caching) for every candidate solution, i.e., at fitness evaluation time. Despite its theoretical simplicity, this approach is not feasible for large systems for two main reasons. First, properly measuring the building time would require to perform the build stage multiple times to have a reliable measure. Second, each building run requires almost one hour for our industrial system.

To overcome these limitations, we *approximate the building cost* by considering on (1) the number of module cache breaks (which determines the number of modules to rebuild) and (2) the size of the module being modified. More precisely, we *estimate the cost of module cache breaks as follows*:

$$Cost(S) = \sum_{M_i \in M} \mathcal{F}_{LOC}(M_i) * Cb(M_i) \qquad (2)$$

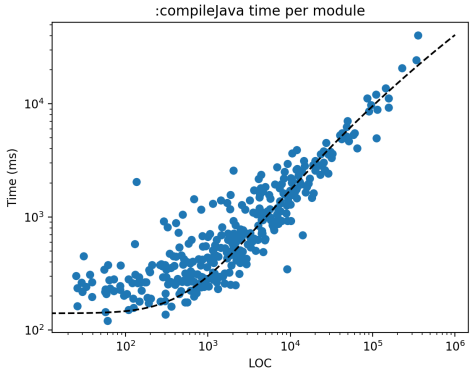

Fig. 1: The runtime of the compilation time and LOC per modules, and the regression function.

where S is a candidate solution; M is the set of modules in the system; $\mathcal{F}_{LOC}$ translates the size of the module (lines of code) into the corresponding build cost; $Cb(M_i)$ is the number of times the module cache is broken based on the rate of changes for the module M_i itself and all its transitive dependencies.

$\mathcal{F}_{LOC}$ approximates the build cost of a module using a linear regression model that uses the lines of code (LOC) of a module and predicts its build time. We note that using $\mathcal{F}_{LOC}$ the estimated build time can be calculated quickly and can be easily re-evaluated after changes are made by simply subtracting the LOC of the moved classes from their original modules and adding it to the ones they were moved to. We show the linear-regression relation between time and LOC of a module and our regression function in Figure 1.

C. The search algorithm

The Non-Dominated Sort Genetic Algorithm II (NSGA-II) [40] was chosen to optimize this problem. This choice is supported by the fact that the number of search objectives to optimize is low (< 5). Previous studies have shown that NSGA-II is the most effective algorithm for software re-modularization and similar problems [15, 21, 20, 23, 17, 14, 18, 16].

NSGA-II is a multi-objective evolutionary algorithm that iteratively optimizes an *initial pool* of N randomly generated solutions, i.e., sets of move-class operations in our cases. In each iteration, the fittest solutions are selected for using the *binary tournament selections*. Selected pairs of solutions are genetically recombined using *crossover* and *mutation*. Newly generated solutions —often referred to as to *offspring*— are evaluated to compute the objective scores. Finally, the N *parents* and N *offspring* solutions are inserted within the same pool, from which only N solutions survive to the next generation. The N solutions to survive are selected using the *fast non-dominated sorting* and the *crowding distance* [40]. The sorting aims to promote solutions that are more optimal than others based on the *Pareto optimality* [40]. The crowding distance aims to promote well-diversified solutions within the phenotype space. The iterative process terminates when the maximum number of iterations is reached [40].

In the following, we described the main elements of NSGA-II that we customized to our problem.

Population Initialization. To generate the initial population, we create a pool of randomly generated solutions. Each random solution S is created by creating k move-class operations (delta changes). The number k is randomly generated within the interval $[1; 100]$. In other words, our initial population will have solutions with different numbers of move-class operations.

Crossover. We propose a crossover operator that aims to preserve building blocks, which in this case are multiple changed classes that only have a positive effect when moved together. This operator is based on the crossover operator proposed by Harman et al. [22].

The *uniform crossover* creates two offspring solutions by exchanging genes (move-class operations) between the two selected parents. The crossover does not exchange classes within the same module across the two parents to preserve building blocks. Instead, it copies all classes withing the same module all together into of the two offspring. This means that all classes in the same module of one parent are all copied in the same in one child, and the classes in the same module in the other parent in the other child. This is repeated for each module, pairing the two children with the two parents randomly for each.

Mutation. We propose two mutation operators: (1) the *min-cut-based operator* and (2) the *neighborhood-based operator*.

The *min-cut-based operator* picks a random class C and builds its dependency graph. In such a graph, the nodes are classes, while the oriented edges denote that one class A (source node) depends on another class B (target node). The graph is built by first adding the randomly selected class C and then all classes on which C is directly control dependent. This process is repeated to add also the classes C is transitively dependent on. The graph build process ends when a certain *depth* is met, which is limited due to the computation cost associated with the min-cut. The mutation operator then performs the Stoer-Wagner algorithm to determine the min-cut of this graph [41]. The initially chosen class is in one of the resulting sub-graphs. All classes withing the sub-graph are then moved to a different module. This operator is inspired by existing research that used graph cutting algorithms for modularization [38, 25, 29].

The *neighborhood-based operator* moves a pool of classes within the same module into another module. This operator was introduced by Fraser and Arcuri [42] for test case generation. The mutation operator was adopted to our problem as follows: it picks a random class, adds it to a set μ, then keeps adding random classes in the neighborhood until the condition $r > (c)^n$ is met. In the condition above, r is a random number between 0 and 1; c is a constant between 0 and 1 and impacts the average size of the resulting set; n is the size of the set. The neighborhood of a class C corresponds to all classes that are in the same module of C and that have at least one structural dependency with the classes in the set μ. This set is then moved to a different module.

D. Circular dependencies

Solutions that contain any circular dependency between two modules (e.g., module A depends on module B, and module B depends on module A) are invalid.

During the design phase, we experimented with punishing and repairing solutions that violates the constraints (circular dependencies) similar to work of Deb et al. [40]. However, we have empirically observed that solutions with circular dependencies had naturally higher fitness values given their larger number of transitive dependencies. Therefore, we decided not to apply any punishment in case a solution contains a circular dependency. We nevertheless remove any possible solution with circular dependencies from the final set.

E. Parameter Tuning

We identify the best hyper-parameters after running the algorithm in a reduced version of the code base that contained 17 modules and around 10k classes. We selected the 17 modules manually as to increase their generalizability towards the entire code base.

We make use of default values based on existing research and the intuition we built throughout its development. Given the large number of possible combinations and the computational cost required to run them all, we opt for tuning a smaller set of parameters, and to tune one parameter at a time while using a default value for the others. We run each configuration three times and picked the one with highest average value.

The best parameter values are as follows:

- Population size = 500.
- Mutation probability $p_m = 0.50$.
- The two mutation operators (*min-cut-based operand* and *neighborhood-based*) have equal (and mutual exclusive) probability to be applied. In other words, each operator has 50% chance to be applied.
- Duplicates are deleted.
- The crossover operator that preserves building blocks is used with a crossover probability $p_c = 1.00$.
- An elite archive with the same size as the population is used.

V. Study I: Effectiveness of the approach

We set out our first research goal to measure the effectiveness of the proposed search-based algorithm in identifying solutions that would improve *Adyen*'s code base. To that aim, we propose two research questions:

RQ_1: *How effective is our approach in producing Pareto-efficient trade-offs that optimize the search objectives?*

RQ_2: *How fast does our approach converge?*

We run our approach on the entire *Adyen* code base for 24 hours on 8 cores at 2.1GHz with 8GB of RAM, two times, one for 24 hours, and one for 72 hours. We could not run NSGA-II more than once for each different search budget due to constraints imposed by the industrial context.

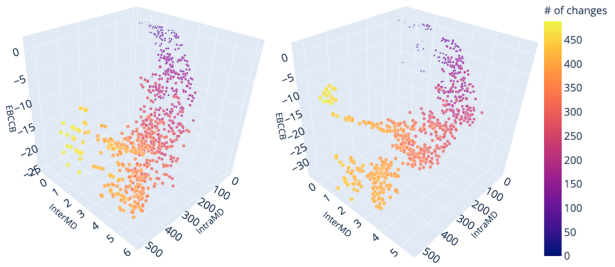

Fig. 2: Representation of the Pareto front of the first (left) and the second (right) runs.

Although multiple runs may generate slightly different results, we performed preliminary experiments for parameter tuning (see Section IV-E). We observed good stability of the results in our preliminary runs.

1) Performance metrics: Many performance metrics have been proposed in the literature to assess the optimality and distribution of the produced trade-offs [43]. The most common performance metrics include hyper-volume (HV) [44], inverted generational distance (IGD) [45], spread [46], and R^2 [47]. However, these metrics requires knowing the true optimal Pareto front and measuring some quality aspects of the non-dominated front generated by NSGA-II (or any many-objective search algorithm). Some quality metrics compute the distance between the generated and true Pareto fronts (generational distance), diversity of the solutions in the front (spread), etc. However, in our context, the true Pareto front is unknown; thus, existing metrics like HV cannot be computed.

Another potential option could consist of creating a hybrid front (also called *reference front*) by combining the non-dominated fronts obtained by using multiple MOEAs, and over multiple runs. This was also not a viable solution as we could not run multiple MOEAs nor performing more repetitions due to the industrial context constraints.

Based on the observation above, we answer RQ_1 by analyzing the maximum improvements that can be obtained in the search objectives. This allows us to measure the best improvements in the software quality metrics, i.e., the extreme points of the produced Pareto front. This also corresponds to the *regions of interests* (ROI) of our industrial partner, whose goal was to understand the extreme alternatives (trade-offs) across the selected search objectives. To answer RQ_2, we analyze how the best improvements in the software quality metrics change over time. More precisely, we analyze the extreme points of the Pareto front every 50 generations, based on the 72 hours run.

A. RQ1: How effective is our approach in producing Pareto-efficient trade-offs that optimize the search objectives?

The results of the runs done to answer RQ1 can be seen in Figure 2, which provides a graphical representation of the Pareto front of both runs. Table II reports the relative improvements of the quality metrics (search objectives).

Observation 1: Solutions identify significant improvements, as measured by the optimization metrics. In the most fit solutions (i.e., the Pareto front), we observe improvements of 205.1% in terms of cohesion (IntraMD), 3.6% in terms of coupling (InterMD), and 1.6% in terms of transitive dependencies and cache breaks (EBCCB). If we focus in single search objectives (i.e., disregarding the quality in the other objectives), we observe even larger potential improvements for the specific metrics, such as 530.4% for IntraMD, 6% for InterMD, and 1.6% for EBCCB.

TABLE II: Solutions in the Pareto front. The IntraMD metric does not grow linearly.

	Best IntraMD	Best InterMD	Best EBCCB
Optimizing all variables			
IntraMD	205.1%	151.3%	55.0%
InterMD	3.5%	3.6%	1.2%
EBCCB	0.1%	0.9%	1.6%
# of moves	170	138	67
Optimizing single variables			
IntraMD	530.4%	322.3%	55.0%
InterMD	1.2%	6.0%	1.2%
EBCCB	N/A	-8.1%	1.6%
# of moves	418	244	67

Observation 2: Different optimal solutions need different amounts of changes. We see that the extreme solutions in the front require from 67 up to 170 class moves to be implemented. Interestingly, the solution that maximizes EBCCB seems to need the least amount of changes, while the solution that maximizes IntraMD needed the most. For the IntraMD metric, we observe that solutions that maximize it tend to suggest creating many new small modules (i.e., modules with a few classes only), which explains why optimizing IntraMD requires many class moves. On the other hand, single class moves may already drastically improve the EBCCB metric, e.g., a class move that removes a large number of transitive dependencies in the graph, explaining why solutions with fewer changes are enough to have a high impact in the metric.

B. RQ2: How fast does the proposed algorithm converge?

Figure 3 shows the improvement of quality metric (search objectives) over the generations during the 72-hours run.

Observation 3: The algorithm seems to converge for the InterMD and the EBCCB metrics. The algorithm convergence at generation 19000 for the InterMD metrics; for the EBCCB, the algorithm seems to be mostly converged, only finding minor improvements after generation 14000. Indeed, we can observe that both metrics reach a plateau in Figure 3.

Observation 4: 72 hours was not enough to observe the algorithm converging for the IntraMD metric. We conjecture that the algorithm has not converged yet because the optimal solution for this metric is a structure constructed of modules with one class per module. The mutation operators can easily move a single class to a new module, which keeps improving this metric over generations. This process will

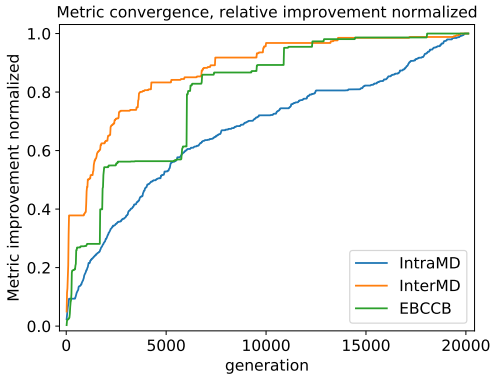

Fig. 3: The fittest solution of the IntraMD, InterMD, and EBCCB over the 72-hours run. Number of changes is not represented in the graph as it is always near zero.

continue until the one class per module structure is achieved. However, this solution would overfit our problem as developers would not accept it. Part of our future work includes avoiding this behavior by, for example, adding more objectives that conflict with IntraMD.

VI. Study II: Developers' Perceptions

As further contribution of our paper, we aim to understand whether the solutions proposed by the approach are considered useful by the developers at *Adyen*. To that aim, we propose another research question:

RQ$_3$: *How do developers perceive the re-modularization suggestions of the approach?*

A. Methodology

We take groups of refactoring suggestions from the (trade-off) solutions produced by NSGA-II in the previous step of this research; then, we present these solutions to the developers at *Adyen* responsible for that module and capture their perception on how useful that particular recommendation is. Given that solutions in the Pareto front improve the entire code base, and our developers work on single modules (and thus, no single developer can review the entire solution), we broke down solutions according to the modules they improve on. For example, suppose any solution X suggests improvements in module A and module B; we consider the refactoring operations related to module A and the refactoring operations related to module B as two possible suggestions to be reviewed by developers. Moreover, we select suggestions that had between 1 to 10 class moves (i.e., a solution small enough that developers could review during their work time) and that improved on at least 2 of the 3 optimization metrics (i.e., IntraMD, InterMD, and EBCCB) and did not worsen the other. We make an exception for the InterMD metric, which was allowed to be worsened if the EBCCB was improved, as InterMD counteracts the process of creating new modules and the EBCCB improvement hints at an improvement in the overall coupling. This process resulted in 13 recommendations to be reviewed by the developers.

TABLE III: The interviewed developers and their experience. Developers that were chosen as seniors to review a suggestion are marked with (s). Experience is represented in years.

Developer	Experience as a developer	Experience at Adyen	Reviewed Suggestions
D1	9	4	C1 (s), C2 (s)
D2	3.5	3.5	C3
D3	10	2.5	C4
D4	10	0.5	C5
D5	20	7	C6 (s)
D6	6	3	C7, C8
D7	3	3	C5
D8	10	2	C9, C10 (s)
D9	34	1.5	C11 (s)
D10	2.5	2.5	C12
D11	6	2.5	C13

We choose a developer who has made multiple changes to the refactoring recommendation classes over the past 6 months to review the suggestions. We leverage *Adyen*'s code repository to extract such information. If no such developer exists or can be interviewed, we choose a senior developer related to that module. This resulted in 11 developers judging 13 different suggestions. Table III reports the experience of the 11 participants on *Adyen*'s code base and in general code development. Four of the developers were seniors instead of developers that made changes to the classes.

During the interview, we asked each participant to review the re-modularization suggestion s/he was assigned to, based on their experience and contributions (see Table III). We further asked the participant to assess the merit of the re-modularization suggestion. If the suggestion was considered good, we asked the interviewee to rate the importance of the change. If the suggestion was considered not good, we then ask the developer to judge whether that class is indeed in the right place and whether it should be moved to another module.

B. Results

In Table IV, we show a summary of the developers' perceptions, whether the identified problematic class is indeed considered problematic, and whether the recommended module for the class to be moved to is considered accurate.

Observation 5: All the 13 suggestions revealed possible improvements to the code base. The developers considered all classes in the 13 suggestions to be currently placed in the wrong module and that they should be moved elsewhere. One exception happens in change C5, where one developer did not consider the recommendation as accurate.

Observation 6: Around half of the suggestions of where to move the classes were deemed to be accurate or partially accurate by developers. Developers labeled seven out of the 13 suggestions about where to move the classes as accurate or partially accurate. Four of these recommendations could be implemented directly. The remaining suggestions were not fully accurate. Among the reasons gave by the developers, we highlight: (i) classes were at the right modules but were poorly

TABLE IV: The developers' perceptions. The first column indicates whether developers considered the identified classes to be problematic. The second column indicates whether the suggestion of modules that classes should be moved to is accurate. Developers fully agree (✓), partially agree (~), or fully disagree (x) with the suggestion.

Change ID	Considered a flaw?	Accurate move suggestion?
C1	✓	x
C2	✓	~
C3	✓	✓
C4	✓	~
C5	✓/ x	✓/ x
C6	✓	x
C7	✓	x
C8	✓	✓
C9	✓	x
C10	✓	x
C11	✓	✓
C12	✓	x
C13	✓	~

designed in terms of coupling or cohesion; (ii) the classes are dead code or superseded by other functionality, so they should be deleted instead; (iii) future plans concerning the classes hinder the suggested change; and (iv) the module the classes are moved to are already in the process of being split up.

Observation 7: Developers did not approve any new proposed modules. Interestingly, developers did not approve any suggestions for creating a new module. Here, we argue that this might be caused by the size of the changes the developers analyzed. Given that the changes were small in size, the suggested new modules were too granular (i.e., contained only a few classes). Therefore, we do not fully disregard the idea of the algorithm proposing new modules, and we will study the developers' perceptions on new modules in future work.

Observation 8: Different developers make different judgments and conclusions. We observe that judging the quality of a recommendation is indeed prone to the developers' perceptions. For example, in the case of C5, two developers gave completely contradictory judgments on the suggestion. D4 determined C5 to be bad and saw no possible improvements, while D7 judged it to be a good suggestion. Moreover, D2 and D6 had similar reasons when judging C3 and C7, but end up with different conclusions. In both cases, the suggestion would be good in the current state of the code, but given the company's future plans related to the feature that the module implements, D2 concluded that the refactoring should be implemented and reverted when needed, whereas D6 concluded that it should not be implemented. Overall, this shows that deciding whether or not a suggestion is good depends on several factors. While optimizing numbers helps developers understand the overall benefit that the change will bring to the code base, in future work, we intend to study the decision-making process to better support developers in such challenging decisions.

VII. Research Challenges and Opportunities

Our results show that search-based re-modularization approaches scale to enterprise-level code bases, and solutions show significant relative improvements. Nevertheless, we still see room for improvements and potential research directions. In the following, we highlight the challenges and opportunities we observed throughout this research, hoping that they will pave the road for future research.

Challenge 1: The need for better cohesion metrics. Despite IntraMD being the most used search objective to improve the cohesion of software modules, in practice, we observed that the algorithm can "overfit" by moving a single class to a new module multiple times, increasing the metric value significantly without actually improving the codebase. This also means that the metric also will not converge until a solution of one class per module has been constructed. For these reasons, we now believe that IntraMD does not accurately represent a given module or code base's cohesion. We, therefore, suggest future work to explore better cohesion metrics. We conjecture that the developers did not accept many move-class recommendations due to this type of overfitting.

In our preliminary study, we have explored the Common Closure Principle (i.e., classes that change together are packaged together) and the Common Reuse Principle (i.e., classes that are used together are packaged together), as proposed by Martin [39], and so far experimented solely by Abdeen et al. [15]. We observed fewer improvements during the parameter tuning phase, and thus, we discarded them in that stage. Future studies should investigate whether the solutions we obtain using these metrics while providing less relative improvements, do in fact, improve the cohesion of modules.

Finally, given the advances of natural language processing techniques and the similar statistical properties that source code hold [48], we suggest future work to explore whether semantically similar classes should also be packed together. We observe researchers taking the first steps towards such an idea (i.e., Ouni et al. [16], Mkaouer et al. [30], Mahouachi [18]); we reinforce the need for such approaches.

Challenge 2: The need for more precise measures of effort. The effort (and risk) of introducing large changes in large-scale software systems should always be taken into account in any provided recommendation. In this research, we used the number of move-class operations as a proxy for effort. This may not properly represent the effort required for the changes, as some classes might be significantly harder to move than others. For example, moving a class to a different module may imply re-designing the test code base, transferring the code ownership to a new team, etc.

We argue that more realistic metrics to measure the cost of applying the refactoring in practice is needed. We identified a single attempt to improve the effort measurement (i.e., the MoJoFM metric [49], which has been used by Bavota et al. [20]). We suggest future work to explore, together

with developers, what metrics best reflect the real effort of performing any re-modularization refactoring.

Challenge 3: The need for more domain knowledge in the process. We have proposed the EBCCB metric to capture the impact of transitive dependencies in *Adyen*'s module cache mechanism. The metric seems to converge. Not many changes are required to achieve significant improvements in the code base. Code change suggestions that mostly improved on the metric were validated by developers.

These results show the importance of adding domain knowledge, which may be company-specific, to the algorithm. We suggest future research to empirically look for other domain-specific functions that can be aggregated to the process. Given that most research in the area is conducted in open-source systems, without real access to developers, we did not find any paper proposing specific search objectives, which we consider as a clear opportunity for future work.

Challenge 4: The need for humans-in-the-loop. We also have observed that relative improvements that the solutions bring to the code base may not be perceived as positive by the developers. We believe that any feedback given by the developers should be fed back to the algorithm. While we have not tried it in this research, we noticed that capturing the developers' perceptions for any proposed recommendation is an expensive task for the company. In this study, developers had to stop working on their main tasks and focus on our research. We envision a more streamlined way of capturing this information in the future. More specifically, we plan to serve recommendations during code review time. Whenever a developer touches a class that is considered to be in the wrong place by our approach, the tool would alert that developer about it. The tool would then ask the developer whether the recommendation is accurate and how important it is to fix it. The tool would show the same recommendation to several developers before not showing it anymore. Their perceptions would then be used back in the next search, i.e., ignore classes that developers consider to be in the correct place.

Integrating these recommendations in their development life cycle would help developers fix the recommendation sooner and provide us with relevant data for more longitudinal empirical studies. We are only aware of the work from Bavota et al. [20], which made use of interactive GA. We suggest future approaches to consider such interaction.

VIII. Threats to Validity

Internal validity. We did not explore every combination of parameters during parameter tuning. Exploring every combination or testing every possible parameter value would result in a vast amount required runs, making the optimization infeasible. In other words, the used combination may not be optimal. More extensive parameter tuning may result in better performance and potentially better solutions from the algorithm.

The suggested groups of class move refactorings that were shown to developers in the interview process are not entirely representative of the solutions they originate from, due to the filtering and grouping process beforehand. Due to this, the conclusion that all suggested groups indicate flaws in the code might not hold for all the class move refactorings that are contained in the entire solutions. Nevertheless, we argue that the portions we show to developers are a significant part of the solution, and thus, serve as a good representative for the entire solutions.

External validity. In regards to external validity, the approach in this paper was applied to a (large) code base which is used and developed by a single company. More research is required to determine whether the results are generalizable to systems using different programming languages, developed by different people, and fulfilling different purposes.

IX. Conclusions

The re-modularization of large-scale software systems is a challenging and expensive task. Developers often have little information about the (positive or negative) impact that changes in modules may have in the software architecture of their systems.

In this paper, we perform a case study on the effectiveness of search-based approaches for software re-modularization at *Adyen*, a large-scale enterprise software system, composed of million of lines of code and hundreds of developers. We also propose a new optimization variable that takes transitive coupling and building time as part of the search problem; a variable that is perceived as fundamental in practice by the *Adyen* developers.

Our results show that such approaches can scale to large code bases and are able to identify modules of the system that require improvements. Nevertheless, recommendations as to which module to move the class to still requires improvement. We hope this paper highlights the importance of such line of research and provides researchers with more insights on how these techniques currently behave in large-scale industrial software systems.

References

[1] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," in *Pioneers and Their Contributions to Software Engineering*. Springer, 1972, pp. 479–498.

[2] G. Booch, R. A. Maksimchuk, M. W. Engle, B. J. Young, J. Connallen, and K. A. Houston, "Object-oriented analysis and design with applications, third edition," *ACM SIGSOFT Software Engineering Notes*, vol. 33, no. 5, pp. 29–29, Aug. 2008.

[3] E. Evans, *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.

[4] S. Newman, *Building microservices: designing fine-grained systems.* " O'Reilly Media, Inc.", 2015.

[5] G. J. Myers, "Composite/structured design," *Google Scholar Google Scholar Digital Library Digital Library*, 1978.

[6] E. B. Allen, T. M. Khoshgoftaar, and Y. Chen, "Measuring coupling and cohesion of software modules: an information-theory approach," in *Proceedings Seventh International Software Metrics Symposium*. IEEE Comput. Soc, 2001.

[7] S. Mancoridis, B. S. Mitchell, C. Rorres, Y. Chen, and E. R. Gansner, "Using automatic clustering to produce high-level system organizations of source code," in *Proceedings. 6th International Workshop on Program Comprehension. IWPC'98 (Cat. No.98TB100242)*. IEEE Comput. Soc, 1998.

[8] G. Bavota, A. D. Lucia, A. Marcus, and R. Oliveto, "Using structural and semantic measures to improve software modularization," *Empirical Software Engineering*, vol. 18, no. 5, pp. 901–932, Sep. 2012.

[9] L. Xiao, Y. Cai, R. Kazman, R. Mo, and Q. Feng, "Identifying and quantifying architectural debt," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. IEEE, 2016, pp. 488–498.

[10] C. K. Kwong, L. F. Mu, J. F. Tang, and X. G. Luo, "Optimization of software components selection for component-based software system development," *Computers & Industrial Engineering*, vol. 58, no. 4, pp. 618–624, May 2010.

[11] M. Kessentini, W. Kessentini, H. Sahraoui, M. Boukadoum, and A. Ouni, "Design defects detection and correction by example," in *2011 IEEE 19th International Conference on Program Comprehension*. IEEE, Jun. 2011.

[12] B. S. Mitchell and S. Mancoridis, "On the automatic modularization of software systems using the bunch tool," *IEEE Transactions on Software Engineering*, vol. 32, no. 3, pp. 193–208, Mar. 2006.

[13] K. Praditwong, M. Harman, and X. Yao, "Software module clustering as a multi-objective search problem," *IEEE Transactions on Software Engineering*, vol. 37, no. 2, pp. 264–282, Mar. 2011.

[14] M. W. Mkaouer, M. Kessentini, M. Ó. Cinnéide, S. Hayashi, and K. Deb, "A robust multi-objective approach to balance severity and importance of refactoring opportunities," *Empirical Software Engineering*, vol. 22, no. 2, pp. 894–927, Mar. 2016.

[15] H. Abdeen, H. Sahraoui, O. Shata, N. Anquetil, and S. Ducasse, "Towards automatically improving package structure while respecting original design decisions," in *2013 20th Working Conference on Reverse Engineering (WCRE)*. IEEE, Oct. 2013.

[16] A. Ouni, M. Kessentini, H. Sahraoui, and M. S. Hamdi, "Search-based refactoring: Towards semantics preservation," in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, Sep. 2012.

[17] A. Ghannem, M. Kessentini, M. S. Hamdi, and G. E. Boussaidi, "Model refactoring by example: A multi-objective search based software engineering approach," *Journal of Software: Evolution and Process*, vol. 30, no. 4, p. e1916, Nov. 2017.

[18] R. Mahouachi, "Search-based cost-effective software remodularization," *Journal of Computer Science and Technology*, vol. 33, no. 6, pp. 1320–1336, Nov. 2018.

[19] A. Fadhel, M. Kessentini, P. Langer, and M. Wimmer, "Search-based detection of high-level model changes," in *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, Sep. 2012.

[20] G. Bavota, F. Carnevale, A. D. Lucia, M. D. Penta, and R. Oliveto, "Putting the developer in-the-loop: An interactive GA for software re-modularization," in *Search Based Software Engineering*. Springer Berlin Heidelberg, 2012, pp. 75–89.

[21] V. Alizadeh, M. Kessentini, W. Mkaouer, M. Ocinneide, A. Ouni, and Y. Cai, "An interactive and dynamic search-based approach to software refactoring recommendations," *IEEE Transactions on Software Engineering*, pp. 1–1, 2018.

[22] M. Harman, R. Hierons, and M. Proctor, "A new representation and crossover operator for search-based optimization of software modularization," ser. GECCO'02. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002, p. 1351–1358.

[23] D. F. D. Silva, L. F. Okada, T. E. Colanzi, and W. K. G. Assunção, "Enhancing search-based product line design with crossover operators," in *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*. ACM, Jun. 2020.

[24] S. A. Ebad and M. Ahmed, "Software packaging approaches —a comparison framework," in *Software Architecture*. Springer Berlin Heidelberg, 2011, pp. 438–446.

[25] C. Jermaine, "Computing program modularizations using the k-cut method," in *Sixth Working Conference on Reverse Engineering (Cat. No.PR00303)*. IEEE Comput. Soc, 1999.

[26] J. K. Lee, S. J. Jung, S. D. Kim, W. H. Jang, and D. H. Ham, "Component identification method with coupling and cohesion," in *Proceedings Eighth Asia-Pacific Software Engineering Conference*. IEEE Comput. Soc, 2002.

[27] B. S. Mitchell, "A heuristic search approach to solving the software

[29] A. Marx, F. Beck, and S. Diehl, "Computer-aided extraction of software clustering problem," 2002, aAI3039424.

[28] B. S. Mitchell and S. Mancoridis, "On the evaluation of the bunch search-based software modularization algorithm," *Soft Computing*, vol. 12, no. 1, pp. 77–93, Jun. 2007.
components," in *2010 17th Working Conference on Reverse Engineering*. IEEE, Oct. 2010.

[30] W. Mkaouer, M. Kessentini, A. Shaout, P. Koligheu, S. Bechikh, K. Deb, and A. Ouni, "Many-objective software remodularization using NSGA-III," *ACM Transactions on Software Engineering and Methodology*, vol. 24, no. 3, pp. 1–45, May 2015.

[31] T. M. Meyers and D. Binkley, "An empirical study of slice-based cohesion and coupling metrics," *ACM Transactions on Software Engineering and Methodology*, vol. 17, no. 1, pp. 1–27, Dec. 2007.

[32] A. Marcus, D. Poshyvanyk, and R. Ferenc, "Using the conceptual cohesion of classes for fault prediction in object-oriented systems," *IEEE Transactions on Software Engineering*, vol. 34, no. 2, pp. 287–300, Mar. 2008.

[33] M. Hitz and B. Montazeri, "Chidamber and kemerers metrics suite: a measurement theory perspective," *IEEE Transactions on Software Engineering*, vol. 22, no. 4, pp. 267–271, Apr. 1996.

[34] L. C. Briand, J. W. Daly, and J. Wüst, "A unified framework for cohesion measurement in object-oriented systems," *Empirical Software Engineering*, vol. 3, no. 1, pp. 65–117, 1998.

[35] G. Bavota, B. Dit, R. Oliveto, M. D. Penta, D. Poshyvanyk, and A. D. Lucia, "An empirical study on the developers perception of software coupling," in *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, May 2013.

[36] J. Bansiya and C. G. Davis, "A hierarchical model for object-oriented design quality assessment," *IEEE Transactions on Software Engineering*, vol. 28, no. 1, pp. 4–17, 2002.

[37] R. Mahouachi, M. Kessentini, and M. Ó. Cinnéide, "Search-based refactoring detection using software metrics variation," in *Search Based Software Engineering*. Springer Berlin Heidelberg, 2013, pp. 126–140.

[38] C. Y. Chong, S. P. Lee, and T. C. Ling, "Efficient software clustering technique using an adaptive and preventive dendrogram cutting approach," *Information and Software Technology*, vol. 55, no. 11, pp. 1994–2012, Nov. 2013.

[39] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. USA: Prentice Hall PTR, 2003.

[40] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[41] M. Stoer and F. Wagner, "A simple min-cut algorithm," *Journal of the ACM (JACM)*, vol. 44, no. 4, pp. 585–591, Jul. 1997.

[42] G. Fraser and A. Arcuri, "Whole test suite generation," *IEEE Transactions on Software Engineering*, vol. 39, no. 2, pp. 276–291, Feb 2013.

[43] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. G. Da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on evolutionary computation*, vol. 7, no. 2, pp. 117–132, 2003.

[44] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach," *IEEE transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.

[45] C. A. C. Coello and M. R. Sierra, "A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm," in *Mexican international conference on artificial intelligence*. Springer, 2004, pp. 688–697.

[46] M. Li and J. Zheng, "Spread assessment for evolutionary multi-objective optimization," in *International conference on evolutionary multi-criterion optimization*. Springer, 2009, pp. 216–230.

[47] M. P. Hansen and A. Jaszkiewicz, *Evaluating the quality of approximations to the non-dominated set*. IMM, Department of Mathematical Modelling, Technical Universityof Denmark, 1994.

[48] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 837–847.

[49] Z. Wen and V. Tzerpos, "An effectiveness measure for software clustering algorithms," in *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE, 2004.